Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Aaron Schröder, Alexander Schwarzpaul, Anna Beketova, Asis Hallab, Franz-Eric Sill, Jitu Daba, Jonathan Kurz, Laszlo Lang, Luka Faensen, Mohamed Akdi, Stephanie Ped, Vivian Bass Vega

University of Applied Sciences Bingen, Germany

Last revised September 3, 2026

Contents

Part I.

Analysis Recipes

1. Subfunctionalization, Neofunctionalization and Dosage-Effect Detection (SND)

Purpose

Detect genes whose expression trajectories diverge between duplicated or orthologous copies in a way consistent with subfunctionalization, neofunctionalization, or dosage effects, using the Tensor Omics geometric framework.

Maintainer: Tensor Omics Developers

Required Input Data

- Per-sample expression matrices (e.g. TPM from `kallisto`), one matrix per species, genes × conditions.
- A sample sheet mapping each column to a condition and replicate.
- Gene-family assignments and ortholog/paralog relationships (produced by the prerequisites below).

Prerequisites / Depends on

- *Gene Family Detection* (see chapter 3)

Note

SND compares copies *within* a gene family, so the family grouping and the ortholog/paralog classification must be finished and sanity-checked before you start here.

Step-by-Step Workflow

Step 1. Normalize each species' expression matrix and assemble a TOX dataset (see the normalization and dataset-creation sub-recipes).

Step 2. Restrict the dataset to families that contain at least two copies in the compared lineages.

Step 3. Compute, per family, the geometric divergence of each copy's expression trajectory relative to the family centroid.

Step 4. Classify each divergent copy as subfunctionalized, neofunctionalized, or dosage-affected using the decision thresholds below.

Step 5. Export the per-gene SND table and the diagnostic trajectory plots.

Example script

Runs the full SND analysis on the bundled example data and writes an `snd_results.tsv` table plus per-family trajectory plots. Running it unchanged reproduces the default results referenced in this recipe.

`python/examples/snd_detection.py`

```
# --- input data
    ------------------------------------------------------------
species = {
    "ath": "data/ath_tpm.tsv",    # Arabidopsis thaliana
    "aly": "data/aly_tpm.tsv",    # Arabidopsis lyrata
}
sample_sheet = "data/samples.tsv"

# --- parameters (see the parameter table below)
    ----------------------------
min_family_size    = 2
divergence_cutoff = 0.35      # angular divergence from the family
    centroid
dosage_log2fc      = 1.0      # |log2 fold-change| flagged as a dosage
    effect
```

Listing 1.1: Fragments to edit: input paths and thresholds

Important Parameters

Parameter	Controls	When / which values
`min_family_size`	Smallest number of copies a family must have to be tested	Keep at 2 for pairwise comparisons; raise to focus on larger, higher-confidence families
`divergence_cutoff`	Angular divergence from the family centroid above which a copy is called "divergent"	Lower (~0.2) for sensitive screening; raise (~0.5) to report only strong divergence
`dosage_log2fc`	Absolute log2 fold-change flagged as a dosage effect	Use `1.0` (2×) as a default; tighten for deep, low-noise data

Expected Results

After step 3 you should have one divergence score per copy, most near zero. On the example data roughly 5–10% of copies exceed `divergence_cutoff`; a much larger fraction usually indicates a normalization or batch problem rather than genuine biology.

Common Pitfall

Comparing raw TPM across species without a common normalization inflates divergence scores. Always assemble the TOX dataset from consistently normalized matrices before computing divergence.

Pitfall: unbalanced replicates

Families whose copies are measured in different conditions produce artefactual trajectory divergence. Restrict to the shared condition set, or impute with care, before classification.

Interpretation

A copy with high divergence but a conserved expression *shape* relative to a sibling suggests *dosage*

change; divergent shape with retained breadth suggests *subfunctionalization*; a novel expression domain absent from all other copies suggests *neofunctionalization*. Read the per-family trajectory plots alongside the table before drawing conclusions.

References

- Robinson, McCarthy & Smyth (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics* 26(1):139–140.
- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

Part II.

Shared Sub-Recipes

2. Normalization of Expression Values

Purpose

Turn a replicate-level expression table into the gene × tissue matrix that every Tensor Omics analysis uses as its expression vectors: gene-wise scale removed, replicates averaged per tissue, dynamic range compressed. This is the shared entry point of the TOX pipeline; the geometry of every later recipe is fixed by the choices made here.

Maintainer: Tensor Omics Developers

Required Input Data

- A table of non-negative expression measurements (e.g. TPM from `kallisto`), genes × replicates, one column per biological replicate.
- The replicate layout: how many replicates belong to each tissue, condition or time point. The replicates of one tissue must occupy *adjacent* columns.
- For stress-response designs only: which tissue is the control and which the condition, for each pair you want a fold change of.

Prerequisites / Depends on

- None — this is the first step of every Tensor Omics analysis.

Step-by-Step Workflow

Step 1. Arrange the measurements as an `(n_replicates, n_genes)` column-major array, so that *one gene is one column*. Group the replicates of each tissue into adjacent rows and record the group sizes in `reps_per_tissue`, e.g. `[4, 5, 5]` for three tissues measured with four, five and five replicates.

Step 2. Decide whether to quantile-normalize across replicates (`use_quantile`). It is off by default; switch it on when the replicate distributions differ enough that the difference is technical rather than biological.

Step 3. Run `normalization_pipeline`. It scales each gene by a LOESS-stabilized standard deviation, optionally quantile-normalizes, averages the replicates of each tissue, and finally applies $x \mapsto \log_2(x + 1)$.

Step 4. Check the result against the expectations below before using it: the shape, the value range, and the number of genes that reached the output unscaled.

Step 5. Stress-response designs only: call `calc_fchange` on the *log-transformed* matrix to turn control/condition pairs into log2 fold changes, so each such axis carries e.g. “drought in root over control root” rather than an absolute level.

Example script

Reads a replicate-level TSV, derives the tissue layout from its header, runs the pipeline and writes the normalized matrix. Run unchanged from the repository root, it normalizes the bundled `material/kallisto_sex_data_no_na.tsv` (88 327 genes, 67 replicates, 13 tissues) in a few seconds and writes `results/normalized_expression.tsv`.

python/examples/normalization.py

```python
import numpy as np
from tensor_omics import normalization_pipeline

# --- input: one gene per COLUMN, -------------------------
#     replicates of a tissue in adjacent ROWS
expr = np.asfortranarray(raw_counts)  # (n_replicates, n_genes)

# --- tissue layout: one entry per tissue, ----------------
#     summing to n_replicates
reps_per_tissue = np.array([4, 5, 5], dtype=np.int32)

# --- parameters (see the table below) --------------------
normalized = normalization_pipeline(
    expr, reps_per_tissue,
    span=0.7,             # LOESS span, mean-vs-SD fit
    degree=2,             # LOESS polynomial degree
    use_quantile=False,   # quantile normalization
)                         # -> (n_tissues, n_genes)
```

Listing 2.1: Fragments to edit: the input, the tissue layout, the parameters

Note

Tensor Omics stores one expression vector per *column* because Fortran is column-major. A matrix handed over as genes × replicates is not rejected — it is silently interpreted as replicates × genes, and every downstream distance and angle is then meaningless. Check `expr.shape` before you call.

Important Parameters

Parameter	Controls	When / which values
`reps_per_tissue`	How the replicate rows are grouped into tissues; entry *i* is the number of adjacent rows belonging to tissue *i*	Must sum to `n_replicates`, otherwise the call fails with `Array size mismatch`. Replicate counts may differ between tissues
`span`	Fraction of the genes entering each local fit of the mean–SD LOESS curve	Keep the default `0.7`. Lower it only if the mean–SD trend is genuinely non-monotonic and the fit visibly oversmooths; too low a span makes the curve follow noise
`degree`	Polynomial degree of the local LOESS fits	Keep the default `2`. Use `1` for a strongly linear trend or very few genes

Parameter	Controls	When / which values
`use_quantile`	Whether the replicate distributions are forced onto a common distribution after gene-wise scaling	Default `False`. Switch on when replicates differ in distribution for technical reasons; leave off when the between-replicate differences are the signal

Expected Results

The result has shape `(n_tissues, n_genes)` and is read-only — call `.copy()` if you need to modify it. For a non-negative input every value is ≥ 0, since $\log_2(x+1) \geq 0$ there. On the bundled example data the values span $[0, 4.06]$ without quantile normalization and $[0, 4.27]$ with it, and 2 627 of the 88 327 genes (3%) have zero variance across their replicates and therefore reach the output unscaled. A value range wildly different from this, or a much larger unscaled fraction, points at the input layout rather than at biology.

Pitfall: replicates that are not adjacent

`reps_per_tissue` describes *contiguous blocks* of rows, not a grouping by name. If the replicate columns of a tissue are scattered through the table, the pipeline still runs and silently averages the wrong columns together. Sort the columns tissue by tissue first; the example script refuses to continue when it detects a non-adjacent tissue.

Pitfall: genes the standard-deviation fit cannot use

The scaling factor comes from a LOESS curve fitted over all genes, so a gene with zero variance across its replicates carries no mean-versus-SD information. Such genes are left out of the fit *and reach the output unscaled* — they are not dropped and not flagged. Count them, as the example script does. If fewer than five genes have non-zero variance, the whole call fails with `ToxInputError: invalid input arguments`; this is what you get for an all-zero matrix, or for a toy example with too few genes.

Pitfall: expecting quantile normalization by default

Quantile normalization is an *option*, not a pipeline stage: the default is scale → average → $\log_2(x+1)$. Two analyses that differ only in `use_quantile` live in different geometries and must not be compared. Record the flag alongside the results.

Pitfall: fold changes computed on the wrong matrix

`calc_fchange` *subtracts* one axis from another. That is a log2 fold change only if you pass the log-transformed matrix — which is what `normalization_pipeline` returns. Passing raw tissue averages instead yields a plain difference of expression levels that merely looks like a fold change.

Warning

`root_mean_sq_normalization` divides by $\sqrt{\text{mean}(x^2)}$, which is not a standard deviation. It is kept for possible future use and must not be used for normalization; use `normalization_pipeline` or `normalize_by_std_dev`.

Interpretation

After normalization, a gene's coordinates state *where its expression sits across tissues*, not how many transcripts were counted. Gene-wise scaling removes each gene's own expression magnitude, so two genes differing a thousandfold in abundance can be neighbours if their tissue profiles agree; the $\log_2(x+1)$ step then compresses the remaining dynamic range so that no single highly expressed tissue dominates a distance. Distances and angles in this space therefore compare the *shape* of expression across tissues.

Because each choice made above defines a different space, results obtained under different settings answer different questions and generally disagree. Agreement between them is evidence of a robust effect; disagreement is usually informative rather than an error. Report the normalization settings with any result derived from this matrix.

References

- Cleveland (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association* 74(368):829–836.
- Cleveland & Devlin (1988). Locally weighted regression: an approach to regression analysis by local fitting. *Journal of the American Statistical Association* 83(403):596–610.
- Bolstad, Irizarry, Åstrand & Speed (2003). A comparison of normalization methods for high density oligonucleotide array data based on variance and bias. *Bioinformatics* 19(2):185–193.

3. Gene Family Detection

Purpose

Group protein-coding genes across species into gene families so that downstream analyses can compare orthologous and paralogous copies. This is a shared preprocessing step reused by several analysis recipes.

Maintainer: Tensor Omics Developers

Required Input Data

- One protein FASTA per species (primary transcripts only).
- A species list / directory layout expected by OrthoFinder.

Step-by-Step Workflow

Step 1. Run OrthoFinder on the per-species protein FASTAs to obtain orthogroups.

Step 2. Optionally refine large orthogroups with MCL clustering to split fused families.

Step 3. Emit a gene-to-family table used by downstream recipes.

Example script

Wraps OrthoFinder + optional MCL refinement and writes `orthogroups.tsv`. A default run on the example proteomes reproduces the family table used elsewhere in this manual.

`scripts/gene_family_detection.sh`

Important Parameters

Parameter	Controls	When / which values
`inflation (-I)`	MCL granularity when refining orthogroups	Higher (`2.0`–`4.0`) yields smaller, tighter families; lower yields larger, more inclusive families

Common Pitfall

Including multiple splice isoforms per gene inflates family sizes and biases every downstream comparison. Reduce each proteome to primary transcripts first.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.